

Basic (cold recall) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `cold_recall.md` : knowing a fact while the page is open and saying it cold, weeks later, are different skills — only the second one is tested in an interview. This exercise practices it on one real question, at every depth. **If not:** the first cold recall you ever did would be in front of an interviewer.

Write your own answer first, in `practice/cold_recall.md`, then compare.

Question used for this model answer (from Doc01's Interview Topics Preview): "Why does logging beat `print()` in production?"

Basic Version

Approach 1 — the plain answer

Story: the plain, honest answer a nervous first-timer gives — correct, one reason, then stop. **If not:** you'd never learn that a short, correct answer is already a good start.

"Logging beats `print()` because you can turn it up or down without changing the code, and it still writes down what happened even after the program's screen is gone. `print()` just shows text once, on screen, with no levels and no record you can look back at later."

This is a complete, correct 30-second answer — the plain, nervous-first-timer version. It's a little longer than the bare minimum, and it's still in plain, spoken-sounding English — that's fine for a cold-recall answer. It names the claim and gives one solid reason, and nothing else.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a structured answer

Story: the same answer with its real mechanism named (levels, handlers), still inside 30-45 seconds — rung 1 of the Four-Depth Format done properly. **If not:** your answer would sound memorized instead of understood.

"Logging beats `print()` in production because it separates *what* gets recorded from *where* it goes and *how much* you see — `print()` conflates all three into one hard-coded statement. Logging gives you severity levels — `DEBUG` through `CRITICAL` — so you can leave detailed instrumentation in the code permanently and control verbosity through configuration, not code changes. And handlers decouple output destination from the log call itself, so the same line can go quietly to a file in production and loudly to the console while you're debugging locally — without touching a single log statement."

Why this answer works: it's rung 1 of this document's Four-Depth Format, done properly — it leads with the claim in one sentence, then backs it with the *specific mechanism* (levels + handlers) rather than a vague "it's more professional." It uses precise vocabulary (severity levels, handlers, decoupling) that signals real understanding, not memorized phrasing. It stays inside the 30-45 second window instead of turning into a full lecture on the `logging` module — climbing to rungs 2-4 of the Four-Depth Format is a different answer, for a different, deeper question.

What the Basic Version misses: the Basic Version's reasons are correct but generic ("turn it up or down," "writes down what happened") — a strong Intermediate answer names the *actual mechanism* (levels, handlers) behind each of those plain-English claims, which is what turns "I've heard this is true" into "I understand why it's true."

Approach 2 — when the interviewer pushes back

Story: in a real interview the follow-up lands before you've finished feeling nervous. This practices staying anchored to your own claim and climbing one rung deeper, instead of restarting. **If not:** the first pushback would knock you back to the beginning.

The curveball: right after you give the Intermediate answer above, the interviewer doesn't pause — they immediately say: "Okay, but why not just fake severity levels with `print()` and a couple of `if` statements around a global `DEBUG` flag? Isn't that the same thing with less setup?"

Model response, pulling the thread instead of restarting: "It gets you partway there, but it breaks down on two things that flag actually can't fix. First, output routing — with a hand-rolled flag, 'send

warnings to the console but errors to a file *and* a Slack webhook' means writing that branching logic yourself at every call site, or building a mini version of what `logging`'s handlers already do for free. Second, and the bigger one: a global `if DEBUG` flag is process-wide — you can't turn up verbosity for just the `payments` module while leaving everything else quiet, which is exactly the situation you're in when you're chasing one bug in production and don't want to flood the log with noise from every other module. `logging`'s per-logger levels give you that for free; the hand-rolled version needs a flag per module, and now you're maintaining a small config system by hand instead of using the one already in the standard library."

Why this answer works: it doesn't repeat the original claim louder, and it doesn't freeze — it takes the interviewer's specific counter-proposal seriously, finds the concrete thing it can't do, and answers with a mechanism (routing, per-module levels), not just "logging is more standard." It also implicitly concedes the honest part of the challenge ("it gets you partway there") before explaining where it actually breaks — conceding what's true first is more convincing than pretending the counter-proposal has zero merit.

What a weaker answer misses: a weaker answer either just repeats "logging is the standard, professional way to do it" (an assertion, not a mechanism — the interviewer will push again), or gets flustered and says "well, you could do that, I guess, but you shouldn't" with no concrete reason — both read as a candidate who memorized a fact but never actually reasoned about why it's true.

Difference between Approach 1 and Approach 2: Approach 1 answers the original question well once. Approach 2 shows what happens the instant that answer gets challenged — which, in a real interview, is most of the time. The skill isn't a better fact, it's staying anchored to your own claim and climbing exactly one more rung of depth on demand, instead of restarting from scratch or over-explaining everything you know about logging in one dump.

Which one should you actually give in a real interview? Open with something close to the Basic Version's plain wording, or Approach 1 if you're confident — either is a correct 30-second answer. Then don't pre-load Approach 2's curveball response; let the interviewer's actual follow-up (if one comes) tell you which specific thread to pull. Volunteering the "why not just fake it with `print()`" rebuttal before anyone asked reads as over-explaining; having it ready the moment it's asked reads as senior.